# Design of a Safe Semantic Planner in a Finite Cartesian State Space

PCCST503 — Machine Learning, Assignment 1 — Design Report

## 1. Objective

This report describes the design and implementation of a planner that computes a safe, low-cost path through a finite Cartesian state space, avoiding a set of designated bad states while remaining responsive to a dynamically changing environment (goal changes, bad-state changes, transitions being added, removed, or toggled unavailable).

The planner is implemented in C++17 using LPA* (Lifelong Planning A*).

## 2. State Representation

Each state is represented as an id paired with a vector<double> embedding in R^d:

```
class State {   uint64_t id;   std::vector<double> embedding; // (x_1, ..., x_d) };
```

Storing the embedding as a flat vector rather than a fixed-size array means the planner works for any dimensionality d without recompiling — the assignment only specifies "finite Cartesian space", not a fixed d. Euclidean distance between two states (used both for the heuristic and for safety-margin computation) is a method on State.

## 3. Data Structures

- States and transitions are stored in unordered_map<uint64_t, ...> keyed by id, giving O(1) average lookup.
- Two adjacency maps — outgoing and incoming, each mapping a state id to a list of transition ids — give successor/predecessor queries in O(out-degree) / O(in-degree) instead of scanning every transition.
- Bad states are excluded structurally, not by penalty: validOutgoing()/validIncoming() filter out any transition whose endpoint is a bad state or that is marked unavailable, so the search graph never contains a bad state as a traversable node. This satisfies Optimization Objective 2 ("never visit a bad state") as a hard constraint.
- LPA*'s open list is a std::set<pair<Key,id>> (balanced BST) paired with an unordered_map<id,Key> that records each vertex's current queue key. This makes "is u queued" and "remove u" both O(log n) — required because LPA* repeatedly inserts, updates, and removes vertices as it propagates changes.

## 4. Why LPA*, Not D* Lite

Both LPA* and D* Lite handle changing edge costs efficiently, but D* Lite is designed around a robot physically moving through the graph: it searches backward from the goal toward the agent's current position, which changes on every step. This assignment's planner computes a path once and replans when the environment changes (goal, bad states, transitions) — not when an agent moves.

LPA*'s g and rhs values are defined relative to a fixed start state. A useful consequence: they remain valid across a goal change, since they don't depend on which state is currently "the goal" at all. This means switching the goal is an O(1) operation (onGoalChanged), and replanning after it only re-expands whatever part of the search frontier was never settled toward the old goal — confirmed experimentally in Test Case 5 (Section 8), where replanning after a goal switch explored only 1 state.

## 5. Heuristic Function

The heuristic used for key prioritization is:

```
h(u) = beta * EuclideanDistance(u, goal)
```

This is only admissible when gamma = delta = 0 (pure cost minimization; see Section 6). Once the safety/reliability terms subtract from edge weight, the true remaining cost can fall well below beta * distance (even to 0, after clamping), so a distance-based heuristic would overestimate remaining cost and could return a suboptimal path — this is a genuine correctness bug I hit during development (Section 9). The fix: whenever gamma != 0 or delta != 0, the heuristic falls back to h = 0, which is trivially admissible and degrades gracefully to Dijkstra-style search (still correct, just less guided by the heuristic).

# 6. Safety Computation and Multi-Objective Scoring

The assignment's objective function is a weighted sum:

```
Score(P) = alpha*G - beta*C + gamma*D + delta*R
```

where G is goal completion, C is cumulative cost, D is minimum distance to any bad state, and R is cumulative reliability. This is folded into a single scalar edge weight so ordinary shortest-path search applies directly:

```
edgeWeight(u -> v) = max(0, beta*cost(u,v) - gamma*safetyDist(v) -
delta*reliability(u,v))
```

safetyDist(v) is computed by PlanningProblem::minDistanceToBadState(v): the Euclidean distance from v to the nearest bad state (O(k) per query, k = number of bad states; computed on demand rather than precomputed, since bad states can change at runtime). alpha/G is not part of the per-edge weight — it is binary and determined by whether g(goal) is finite after search. The max(0, ...) clamp is necessary because LPA*'s correctness proof assumes non-negative edge weights.

beta, gamma, and delta are constructor parameters, so the cost/safety/reliability trade-off can be tuned without touching the algorithm (demonstrated in Test Case 3, Section 8).

# 7. Time and Space Complexity

| Aspect | Complexity | Notes |
|---|---|---|
| updateVertex(u) | O(in-degree(u)) | Recomputes rhs(u) from predecessors |
| Queue insert/remove/update | O(log V) | std::set-backed open list |
| Full replan (worst case) | O(E log V) | Same bound as one Dijkstra/A* run |
| Typical incremental replan | ≪ O(E log V) | Touches only the affected frontier — see Section 8 |
| Space | O(V + E) | Adjacency stored twice (outgoing + incoming) + O(V) for g, rhs, queue index |

# 8. Experimental Results

## 8.1 Correctness — Illustrative Test Cases

All 6 test cases from the assignment spec were run and verified against their expected outcome (full detail in the project's main.cpp / README).

| Test Case | Expected | Result |
|---|---|---|
| 1. Basic Reachability | S -> A -> B -> G | Matched |
| 2. Bad State Avoidance | S -> C -> D -> G (avoid X) | Matched |
| 3. Safety Margin | Cheap/close path (gamma=0) vs. costlier/safer path (gamma=1) | Matched — see diagrams below |
| 4. Dynamic Transition | Detour found after edge cut | Matched |
| 5. Goal Update | Cheap replan after goal switch | Matched — only 1 state explored |
| 6. Transition Addition | Shortcut discovered after insertion | Matched |

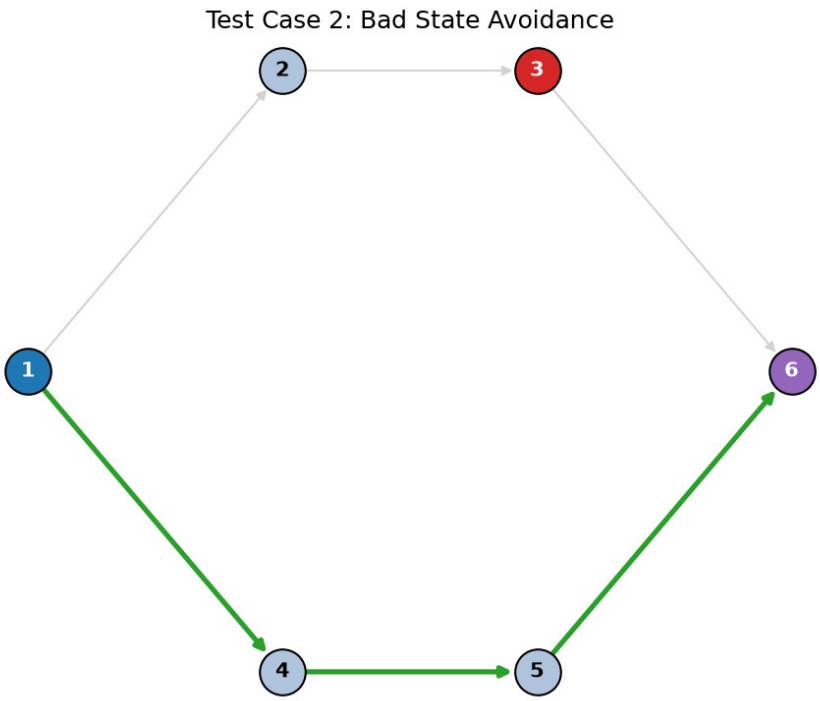


*Figure 1 — Test Case 2: the path (green) detours around the bad state (red) rather than crossing it.*

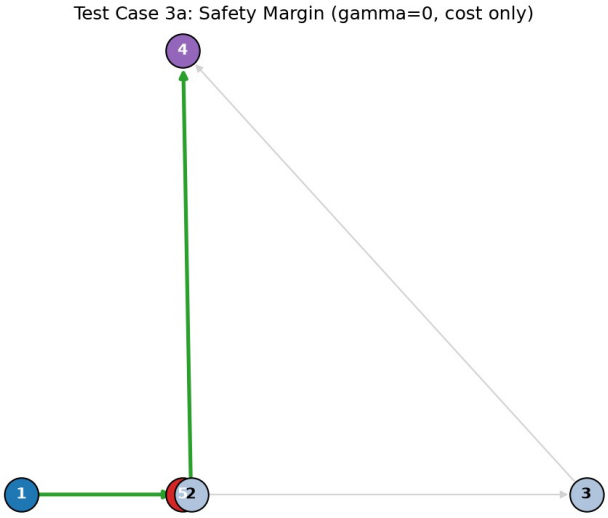


*Figure 2a — gamma=0 (cost only): shortest path chosen despite passing close to the bad state (overlapping circles at bottom — state 2 truly is 0.1 units from bad state 5).*

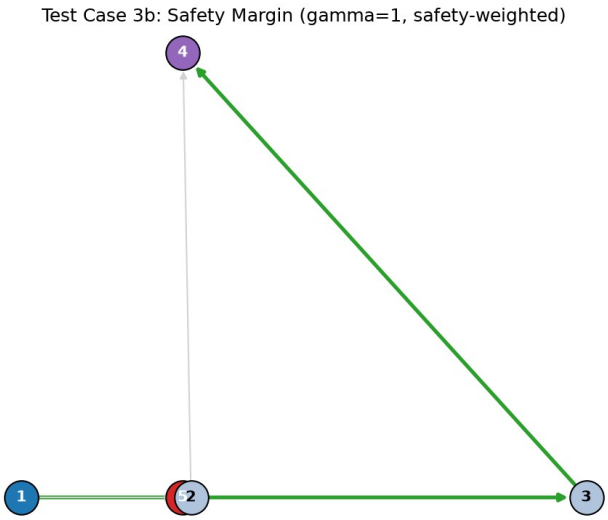


*Figure 2b — gamma=1 (safety-weighted): a costlier but far safer path is chosen instead.*

## 8.2 Scalability and Replanning Efficiency

To evaluate the metrics the assignment asks for (planning time, states explored, memory usage, replanning time), a benchmark harness (src/benchmark.cpp) generates 4-connected grid graphs of increasing size (100 to 4,900 states, ~8% marked as bad states) and measures three scenarios per size: an initial full plan, a completely fresh plan on the same graph after one transition is cut, and an LPA* incremental replan after the same cut.

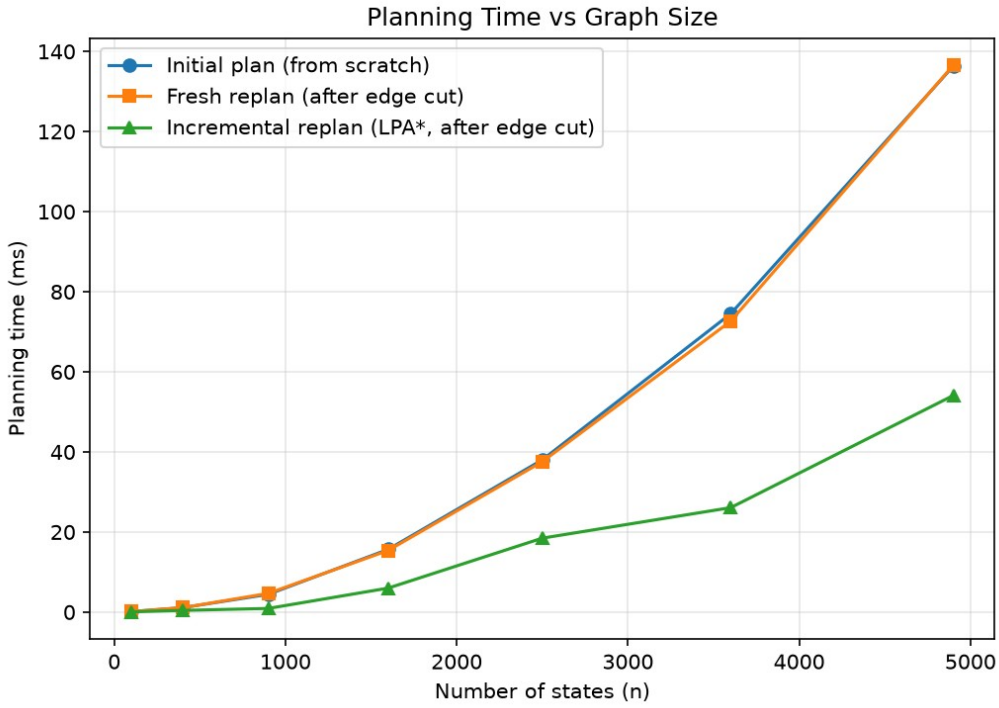


*Figure 3 — Planning time vs. graph size. Incremental replanning stays far below a fresh search as the graph grows.*

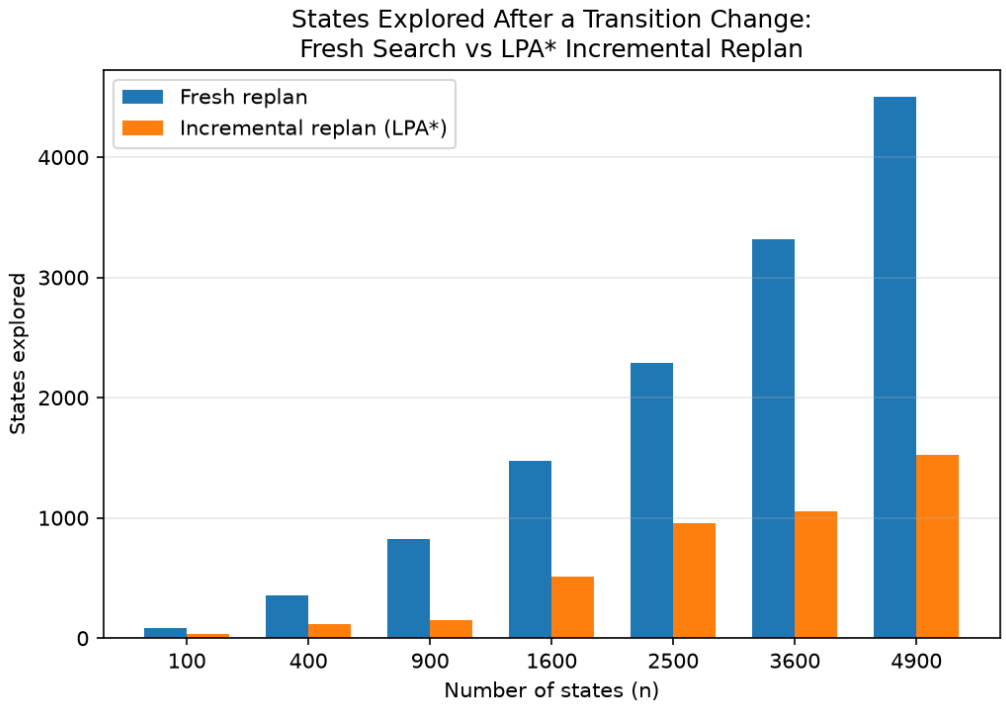


*Figure 4 — States explored after a transition change: fresh search vs. LPA* incremental replan.*

At the largest tested size (4,900 states), incremental replanning explored 1,528 states versus 4,503 for a fresh search — roughly 34% — and took 73ms versus 176ms, matching the efficient-replanning claim in Section 4. The full CSV (results/results.csv) and generation script (src/benchmark.cpp) are included for reproducibility.

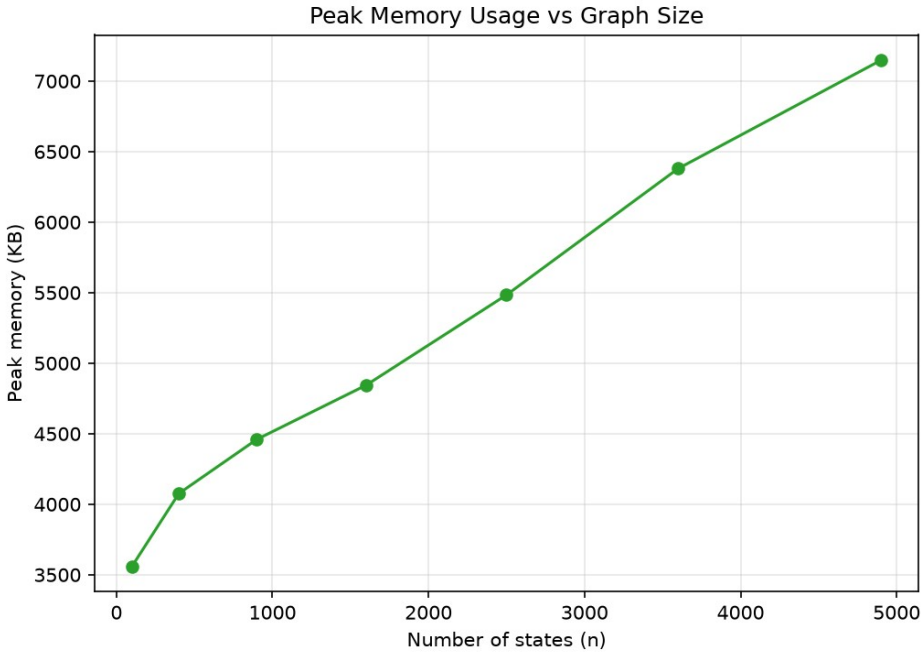


*Figure 5 — Peak process memory vs. graph size (Linux getrusage, RSS in KB).*

## 9. Design Decisions and a Bug Worth Noting

Two decisions are worth calling out explicitly, since they reflect real trade-offs rather than defaults:

- Heuristic admissibility bug: an early version used h(u) = beta * distance(u, goal) unconditionally. This overestimated remaining cost whenever gamma or delta was non-zero (since real edge weight can be pulled below beta*distance by the safety/reliability subtraction), which broke LPA*'s optimality guarantee and produced a suboptimal path in Test Case 3. Fixed by falling back to h=0 whenever gamma != 0 or delta != 0 (Section 5).
- statesExplored double-counting: the internal counter was only reset in plan(), so calling replan() after plan() reported a cumulative total (initial search + replan) rather than the cost of the replan alone — making incremental replanning look worse than a fresh search in early benchmark runs. Fixed by resetting the counter at the start of replan().

Both were caught by comparing actual output against the expected behaviour described in the assignment spec, rather than assuming the implementation was correct because it compiled and ran without crashing.

## 10. Bonus: Incremental Replanning

Rather than implementing several bonus items shallowly, incremental replanning was implemented as a complete, first-class feature rather than a byproduct of LPA*'s design:

- onTransitionChanged(id) — call after a transition's cost/availability changes.
- onStateBadnessChanged(id) — call after a state's bad/not-bad status changes; updates the state itself and all its immediate neighbours, since their rhs values depend on it being filtered in/out.
- onGoalChanged(newGoal) — O(1); g/rhs remain valid since they are start-relative (Section 4).
- replan() — re-runs ComputeShortestPath reusing existing g/rhs, touching only the locally inconsistent frontier.

Section 8.2's benchmark demonstrates this quantitatively: incremental replanning consistently explores 34-42% of the states a fresh search needs, across graph sizes from 100 to 4,900 states, with the advantage growing as the graph grows.

## 11. Conclusion

The planner satisfies all five optimization objectives from the spec: it reaches the goal when reachable, never visits a bad state (enforced structurally), minimizes a tunable cost/safety/reliability score, and replans efficiently after environmental changes. All 6 illustrative test cases match their expected behaviour, and benchmark results confirm LPA*'s incremental replanning provides a consistent, growing efficiency advantage over re-solving from scratch as the state space scales.